Multi-Objective Optimization for Audio Compression

- **Author:** Specialist Prof. Alejandro Lloveras
- **Course:** Evolutionary Algorithms
- **Master's Program:** Artificial Intelligence
- **Chair:** Prof. Miguel Augusto Azar
- **Institution:** Faculty of Engineering, University of Buenos Aires (UBA)

In this project, the optimization of WAV to MP3 audio file compression was proposed based on the Perceptual Evaluation of Audio Quality (PEAQ) and Distortion Index (DI) algorithms, which are audio perception metrics based on a human ear model. The objective was to reduce the file size while sacrificing the least amount of perceptual audio quality.

▸ GitHub Repository

Problem Definition

The use of lossy compression algorithms for multimedia files is complex and requires configuring multiple different parameters that can lead to diverse results. In the specific case of audio, the compression used for MPEG-2 Audio Layer III (popularly known as MP3) offers 6 configurable parameters with 520,290 possible combinations. The quality of the compression depends on the proper selection of these. On the other hand, the human ear's frequency response is extremely complex and exhibits non-linear behavior.

Fletcher-Munson curves, also known as equal-loudness contours, describe how frequencies are perceived at different volume levels. As can be seen in Figure 1, the ear's behavior differs depending on the nature of the source (frequency, harmonic components, bandwidth, timbre, loudness, etc.). It is therefore evident that the ideal compression for a conversation will not be the same as for a music file.

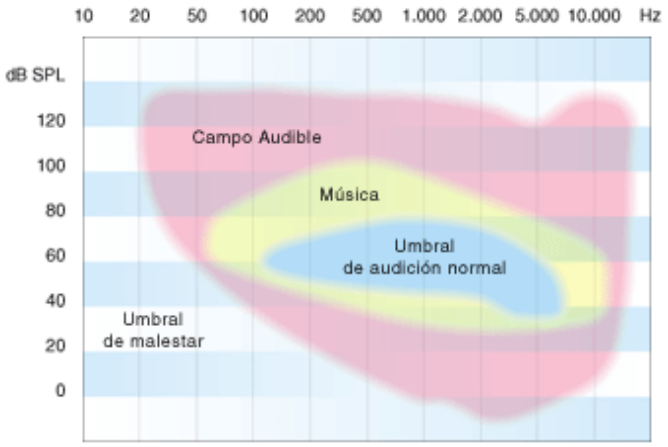


Fig 1: Equal-loudness contours of human hearing.[1]

To find the optimal combination of parameters that minimizes the MP3 file size with the least loss of audio quality, it is necessary to be able to adequately quantify the perception of the human auditory system. This requires building a mathematical model that adequately emulates it. For this work, the *PEAQ* (Perceptual Evaluation of Audio

[1] Image taken from the website http://elruido.com/divulgacion/curso/umbrales.htm

Quality)[2] metric and the *IM* (Improved Metric of Informational Masking)[3] distortion index were used. These metrics range from 0 to -4, where 0 is "maximum quality" and -4 is "extreme loss". Particularly, *IM* can take values even lower than -4, indicating greater deterioration.

Implementation

The main objective is to find an optimal set of audio encoding parameters that minimize the resulting file size and processing time, while maximizing audio quality (measured by *PEAQ* and a distortion index). This is therefore a multi-objective optimization problem. Initially, optimization directions were defined with extreme values: 1 to maximize or -1 to minimize.

Third-Party Libraries

For the solution's implementation, it was necessary to install several third-party libraries. The *FFmpeg open-source framework* was used for multimedia file conversion (controlled via the "ffmpeg-python" library). For emulating the human ear and measuring audio quality (*PEAQ* and *IM*), the "GST Peaq" plugin for the *GStreamer framework* was used, accessed through terminal commands. The *DEAP framework*, an evolutionary computation framework for rapid prototyping and testing of ideas, was used to implement the genetic algorithm.

Development

The programming paradigm used was modular development. Initially, the audio compression and metric calculation modules were developed:

- ***compressor.py:*** It is responsible for the actual audio compression from WAV to MP3 using ffmpeg-python, applying the parameters specified by the evolutionary algorithm.
- ***peaq.py:*** It provides an interface to the external *PEAQ* command-line tool, used for objective measurements of perceptual audio quality. It also manages environment variables for the correct execution of the tool.

In turn, these modules are triggered by:

- ***orchestrator.py:*** It acts as the nerve center of the evaluation process. It calls *compressor.py* to compress the audio and *peaq.py* to evaluate the quality of the compressed file. It also measures the file size and total processing time.

The multimedia files are stored in the *"media"* folder. The work used WAV input files and MP3 output files (which are only temporarily stored for the genetic cycle).

Next, the basic structure for the evolutionary cycle was developed using the NSGA-II (Non-dominated Sorting Genetic Algorithm II) algorithm to optimize the audio compression parameters configured for FFmpeg. In the ***evolution.py*** script, the genes (audio

[2] Kabal, P. (2002). *An Examination and Interpretation of ITU-R BS.1387: Perceptual Evaluation of Audio Quality*. McGill University. https://www.mmsp.ece.mcgill.ca/Documents/Reports/2002/KabalR2002v2.pdf
[3] Delgado, P. M., & Herre, J. (2023). *An improved metric of informational masking for perceptual audio quality measurement*. arXiv preprint arXiv:2307.06656. https://arxiv.org/abs/2307.06656

compression parameters) are defined using DEAP, and the evaluation of each individual is orchestrated by sending these parameters to *orchestrator.py*.

Given the enormous amount of data generated in each evaluation, the need to generate logging files, defined through the ***logger.py*** script and stored in the *"logs"* folder, was detected.

Each of the individuals generated during the evolutionary cycle is stored in a CSV file within the *"history"* directory, storing the FFmpeg parameters and the evaluated objectives. At the end of the evolution, another CSV with the best individuals found with the Pareto front, called Hall of Fame (HOF) in DEAP, is stored within the *"results"* directory.

Finally, the obtained results are evaluated with the help of a Jupyter notebook called ***graph.ipynb***, which facilitates the analysis and visualization of the training cycle.

Genetic Algorithm

NSGA-II (Non-dominated Sorting Genetic Algorithm II) is a popular multi-objective genetic algorithm designed to solve problems with multiple objective functions that are often in conflict with each other. Its main goal is to find a set of solutions that represent the optimal Pareto front, which are solutions where one objective function cannot be improved without worsening at least one other.

Genetic Parameters (Genes)

Each "individual" in the population represents a set of FFmpeg parameters and consists of the following genes:

- **Audio sample rate *(ar):*** An index that maps to a predefined list of common sample rates (8000 Hz to 48000 Hz).
- **Bit Depth or Sample Format *(sample_fmt):*** An index that maps to sample formats like 's16p', 's32p', or 'fitp'.
- **Compression level *(compression_level):*** An integer between 0 (best quality/least compression) and 9 (maximum compression).
- **Reservoir *(reservoir):*** A boolean to enable or disable LAME's bit reservoir control.
- **Encoding mode *(encoding_mode):*** An index that selects between:
    - 0: *audio_bitrate* (CBR - Constant Bit Rate)
    - 1: *aq* (VBR Quality - Variable Bit Rate)
    - 2: *abr* (Average Bitrate)
- **Mode value:** A float value that is interpreted as a bitrate (in kbps) for CBR/ABR or VBR quality (0-9).

Fitness Function

The fitness function, *"evaluate_ffmpeg_params"*, performs the following steps for each individual:

- **FFmpeg Parameter Generation:** The individual's genes are decoded and transformed into a dictionary of parameters for FFmpeg. Constraints and mappings are applied to ensure the validity of the parameters.
- **Execution of the Evaluation:** A function *"evaluate"* (defined in *orchestrator.py*) is invoked, which executes FFmpeg with the generated parameters and measures key metrics of the resulting audio file:
  - *size:* File size in kilobytes.
  - *peaq: PEAQ* (Perceptual Evaluation of Audio Quality) score.
  - *im:* Distortion index.
  - *time:* Processing time.

Problems during Development

FFmpeg Parameters

Given that the documentation for the "ffmpeg-python" library was quite incomplete, numerous tests were necessary through the command line to determine the range and type of configurable parameters for WAV to MP3 compression.

Processing Time

While the file conversion performed by FFmpeg is relatively fast, when performing long training sessions with thousands of individuals, the accumulated file processing times can be considerable, extending training sessions for multiple hours. For example, for a standard 4-minute music track (a 70 MB .wav file with 24-bit depth and 44.1 kHz sample rate), the complete compression and analysis cycle took an average of 11.3 seconds. Training just 2000 individuals would increase the time to over 6 hours.

To minimize the audio processing time, it was decided to use the "simple" *PEAQ* implementation (5.3 sec) instead of the "advanced" one (43.8 sec), as it increased the analysis time by 8.5x. Similarly, small music fragments with durations of 5 and 10 seconds were generated for testing during the development stage. For the actual training, representative 15-second fragments of real music from a musical ensemble (homestudio recording) and 11-second fragments of an orchestra (professional studio recording) were generated. These fragments were carefully selected to contain a wide harmonic and dynamic range.

Audio Quality

The standard CD audio quality of 44.1 kHz and 24 bits is widely used by all commercialized music. It was extremely difficult to obtain lossless files (FLAC or WAV format) that would allow for an effective measurement of the compression's effect. The 15-second fragment (with 44.1kHz and 24bits) was self-authored material recorded previously. The string concert (with 96kHz and 24bits) - an 11-second fragment - was later obtained from a royalty-free music repository in FLAC format. It was decided to work with this fragment because it was of the highest quality, allowing for the analysis of the effect of reducing the sample rate to 48 kHz and comparing it with 44.1kHz.

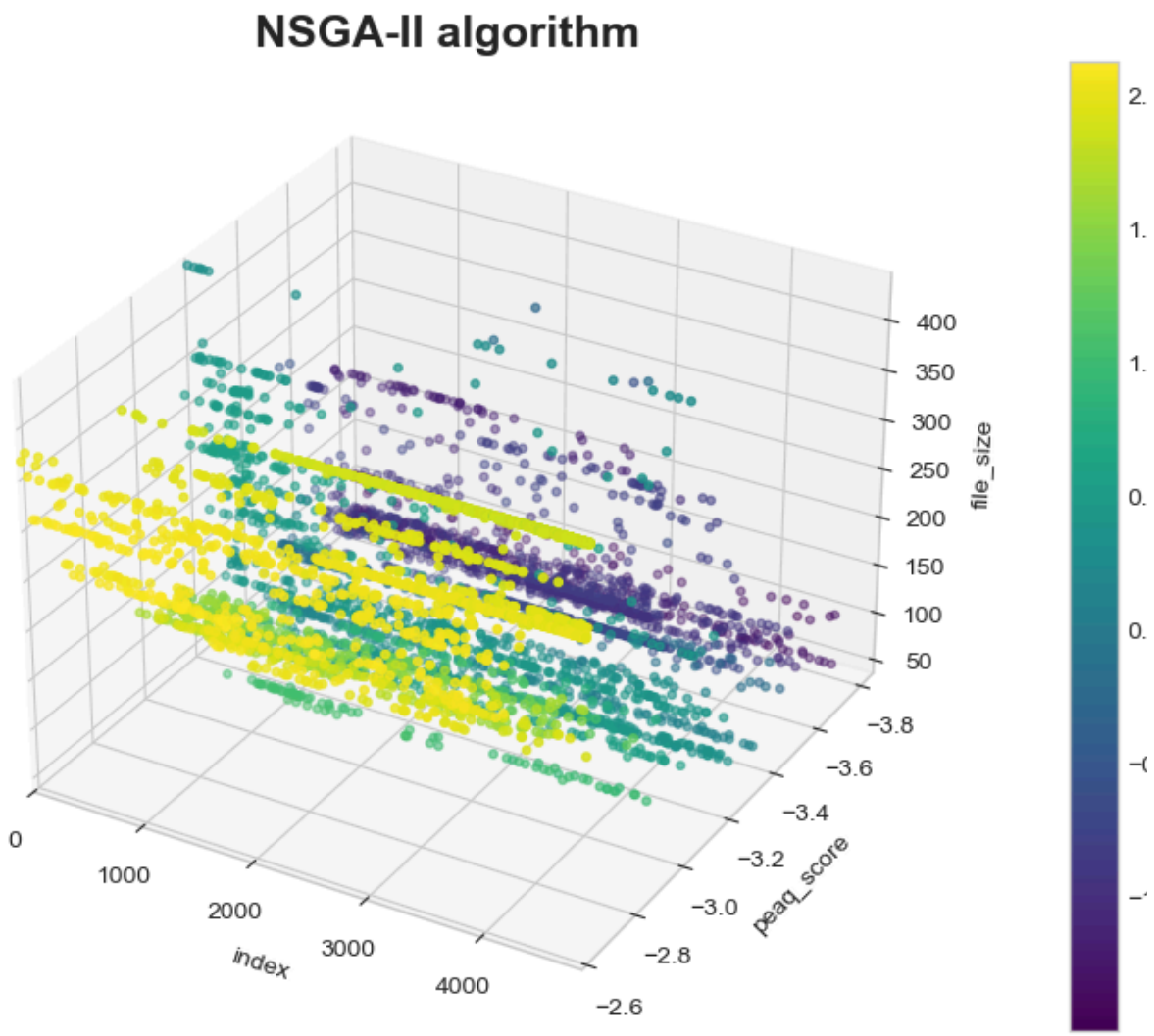


Fig 2. Temporal evolution of *PEAQ* and compression (related to *Fitness*)

Weighting

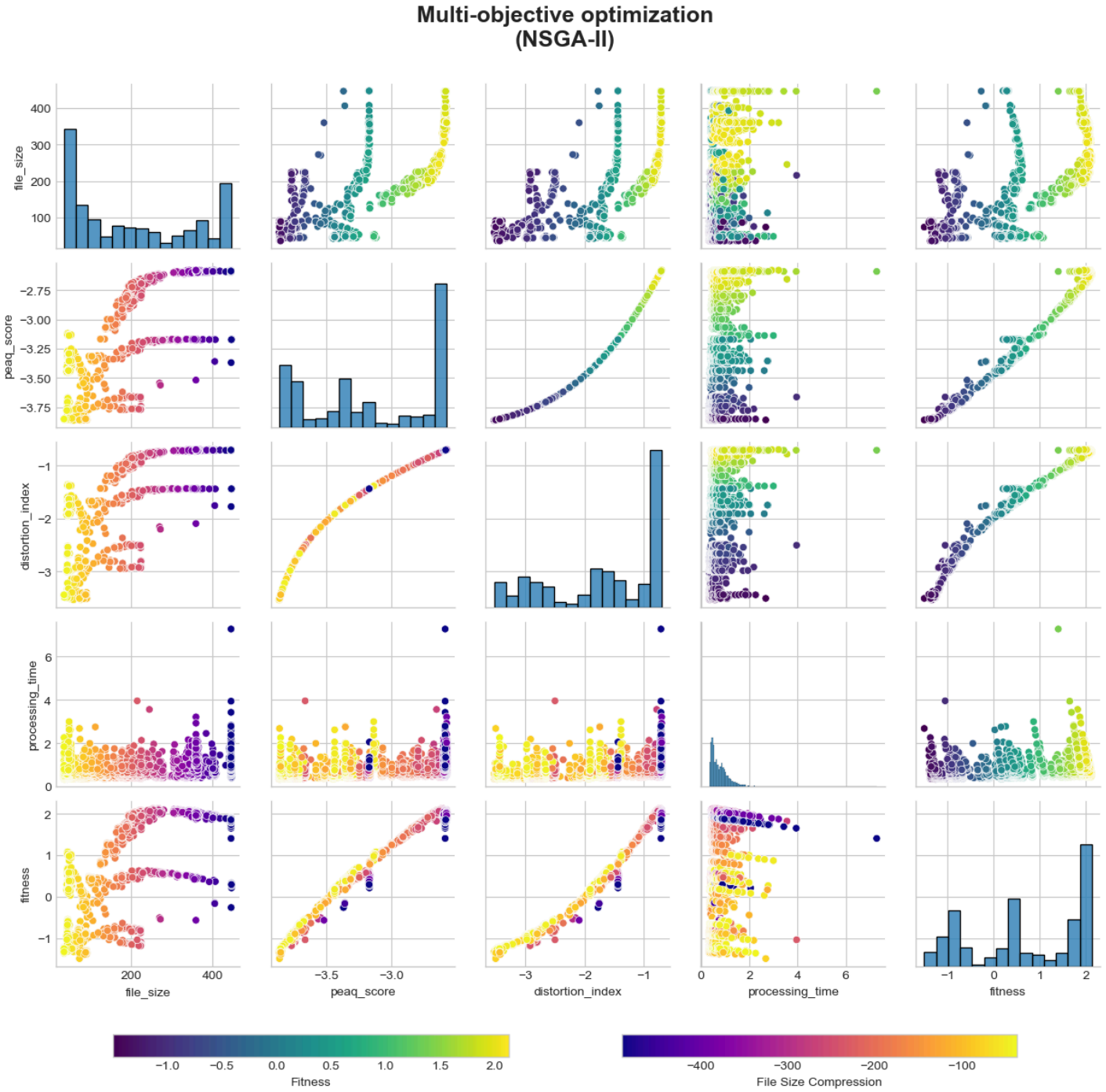
Although initially, optimization direction was defined only, it was soon observed that the evolutionary algorithm always achieved optimization by reducing the sample rate, ignoring the other objectives, even when this harmed quality metrics. It was then decided to assign different weightings to each objective to give greater importance to the *PEAQ* metric than to the file size, since extreme compression was not desirable.

The weightings were manually adjusted until "reasonable" values (according to a specialist's criteria) of sample rate and bit rate were obtained, defined as follows:

1. **100% *peaq*:** quality metric,
2. **50% *size*:** file size in kilobytes,
3. **30% *im*:** distortion index,
4. **5% *time*:** processing time.

Little importance was given to *IM* because it was mostly correlated with *PEAQ* (except in specific cases). For its part, a minimum weight was assigned to the processing time

objective so that, in case of obtaining two individuals with similar quality and size values, the one that required less processing would be chosen.

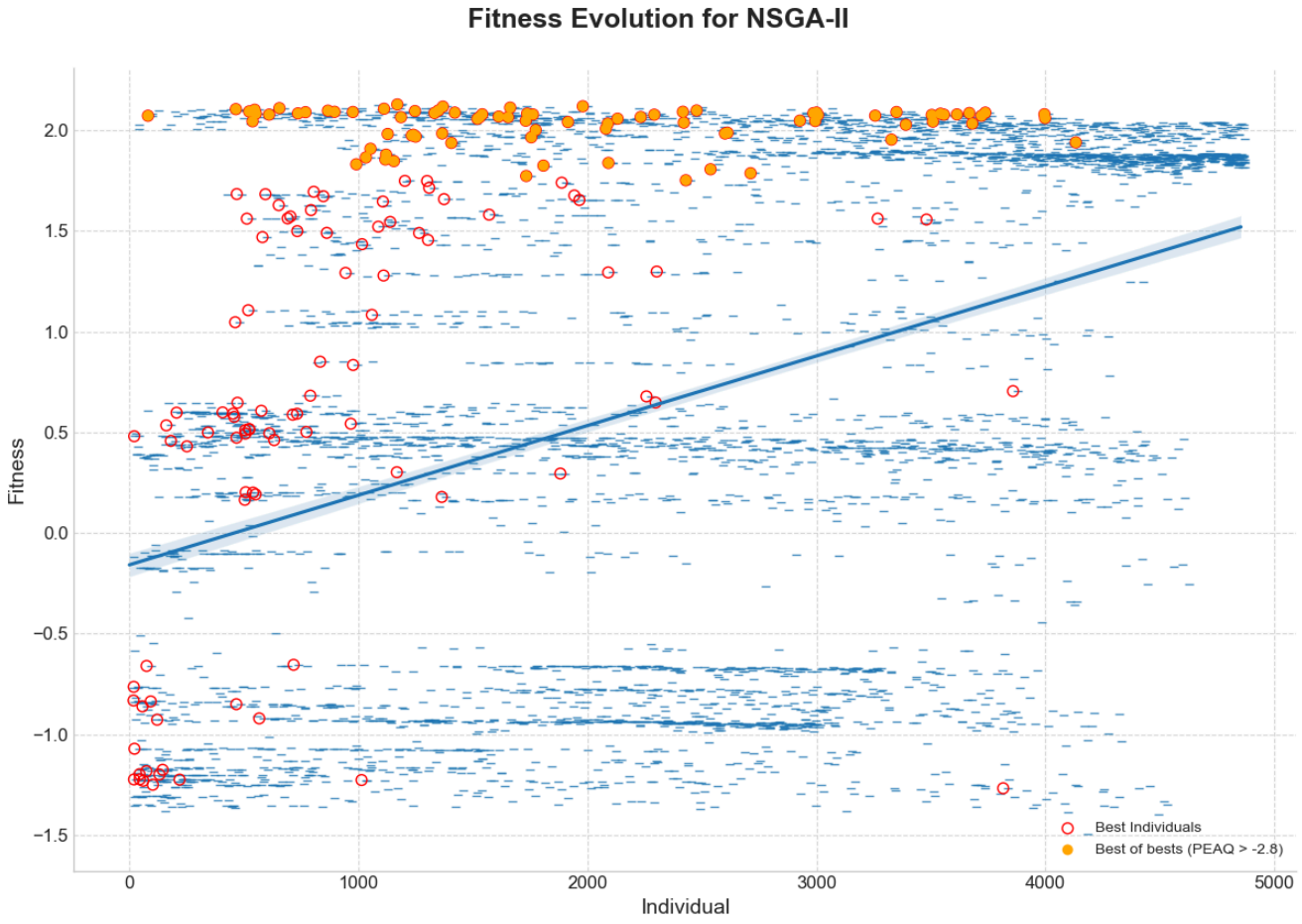


Normalization

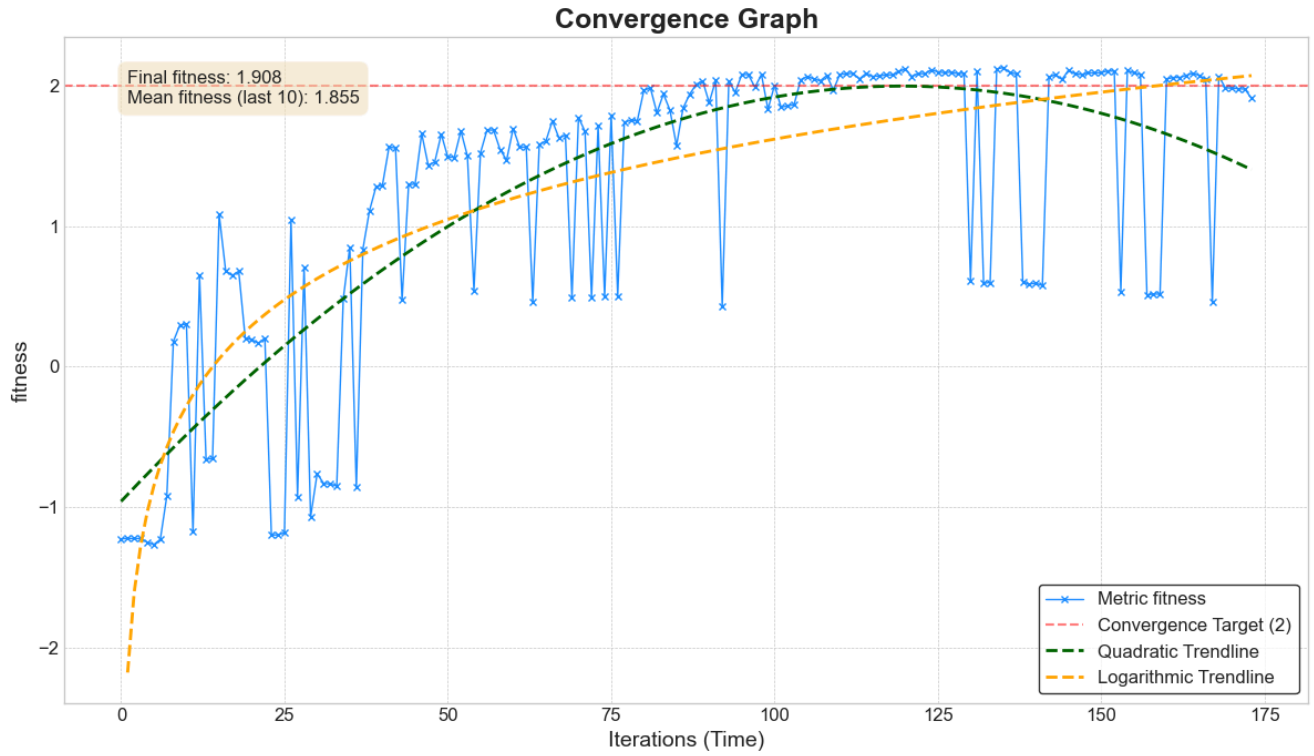
Initially, it was thought that DEAP normalized the objectives, but it was later detected that normalization needed to be applied to the Fitness Function values because they had different magnitudes: file size in kilobytes, processing in milliseconds, and quality metrics in the range of 0 to -4. Z-score normalization was applied using mean and standard deviation values pre-calculated from data stored during previous runs (around 15,000 individuals).

Convergence Graph

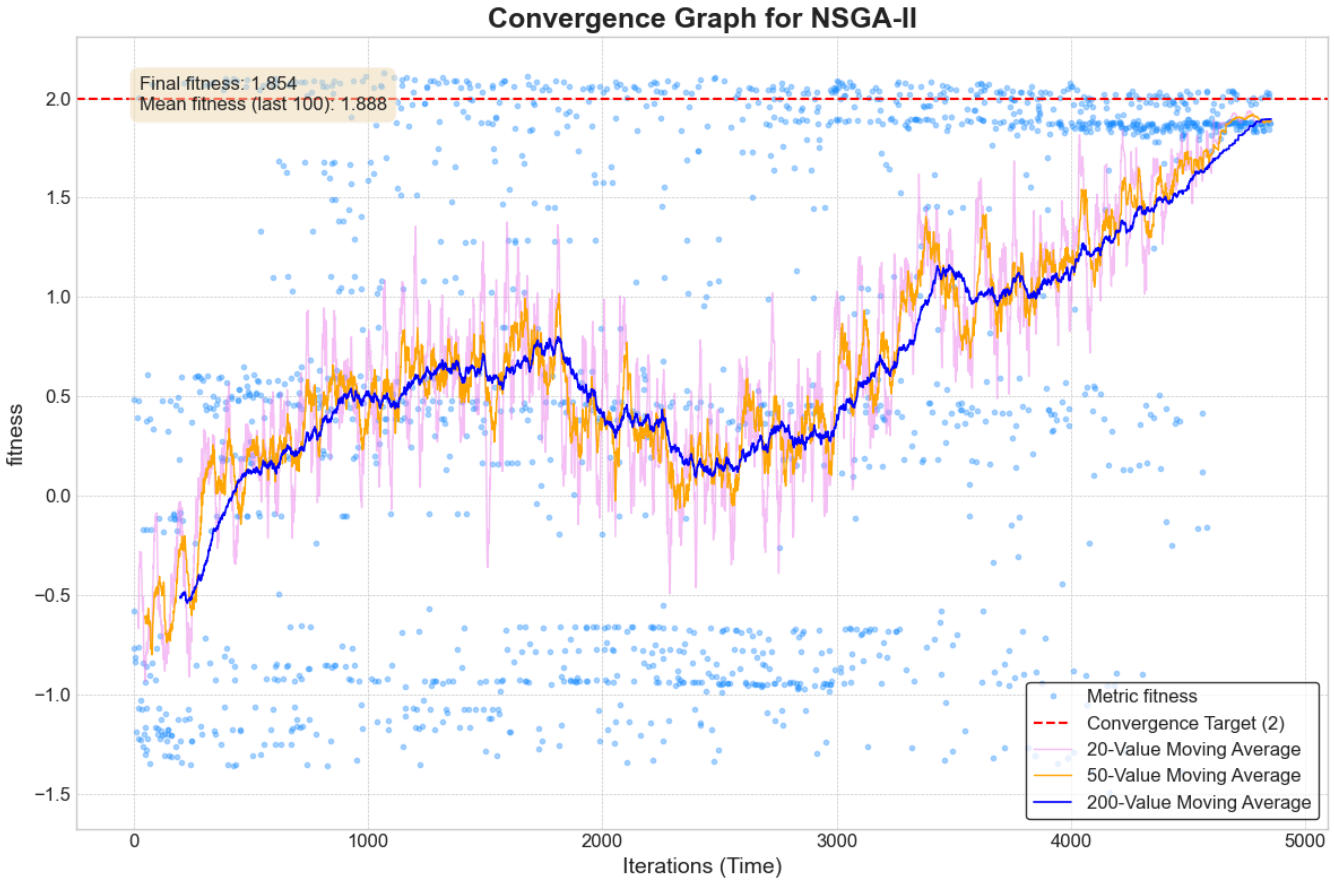
Given the large number of individuals analyzed and the sensitivity of the objective metrics to small changes in the configuration of the FFmpeg compression parameters, the *fitness* metric showed great volatility during training, which made convergence visualization difficult. It was then decided to visualize the evolution through the variation of the average *fitness*. In a first instance, a trend curve was fitted to see if the slope was positive. The individuals belonging to the HOF were also highlighted, and among those, the ones with the best *PEAQ* metrics (threshold -2.8) were selected.



It was also explored to observe only the HOF individuals, plotting quadratic and logarithmic trend lines as a reference to observe their evolution.



Finally, it was decided to work with a set of 3 moving averages of 20, 50, and 200 individuals to observe the general trend and the medium and short-term volatility.



Genetic Operators (NSGA-II)

Through several experimental runs with the NSGA-II algorithm, different genetic operator configurations (population size, number of generations, crossover, and mutation probability) were explored to determine the hyperparameters that maximized the Fitness Function (normalized and weighted multi-objective metrics).

The strategy applied consisted of varying the genetic operators and population parameters one by one, in an ascending and descending manner, to evaluate their impact on the optimization process.

Configuration Parameters

NSGA-II	Base	Mutation ↑	Mutation ↓	Population ↓	Population ↑	Generations ↑	Crossover ↑	Mix
Crossover Probability	80%	80%	80%	80%	80%	80%	95%	95%
Mutation Probability	10%	20%	5%	10%	10%	10%	5%	1%
Population	50	50	50	25	100	50	50	50
Generations	50	50	50	50	50	100	50	100
Individuals	2500	2500	2500	1250	5000	5000	2500	5000

Results

	Base	Mutation ↑	Mutation ↓	Population ↓	Population ↑	Generations ↑	Crossover ↑	Mix
Average *fitness*	1,759	0,840	0,310	1,038	0,302	1,915	0,735	1,888
Winners	115	205	91	103	149	118	205	174
Optima	62	101	37	65	72	57	94	83
Success Rate	54%	49%	41%	63%	48%	48%	46%	48%
Compression	-96,648%	-96,646%	-96,501%	-96,678%	-96,596%	-96,625%	-96,767%	-96,856%
PEAQ Score	-2,628	-2,635	-2,639	-2,636	-2,628	-2,635	-2,645	-2,648
Distortion Index *(IM)*	-0,745	-0,753	-0,758	-0,754	-0,745	-0,753	-0,764	-0,768
					2nd Best			*Best*

It can be seen from the table that, thanks to the hyperparameter adjustment, a greater compression was achieved with similar quality metrics. It was found that:

- For the same number of individuals, increasing the number of generations results in greater improvement than increasing the population.
- Increasing the mutation probability made the exploration more erratic, preventing improvement.

- If the crossover probability was increased, better results were achieved in audio compression (by exploring different combinations of FFmpeg parameters).
- Stagnation at local maxima can be avoided by decreasing the mutation probability.
- By combining a higher crossover probability and a lower mutation probability, the evolution was slower but more controlled (and therefore predictable).

Other algorithms exploration

Subsequently, the NSGA-II result was contrasted against other standard algorithms for multi-objective problems: NSGA-III, SPEA2, PAES, and MOEA-D.

For their execution, the minimum necessary modifications were made to implement the new algorithm, trying to maintain similar numbers of individuals and genetic operators.

Configuration Parameters

	NSGA-II	NSGA-III	SPEA2	PAES	MOEA/D
Crossover Probability	95%	95%	95%	N/A	95%
Mutation Probability	1%	1%	1%	50%	1%
Population	50	50	50	50	165
Generations	100	100	100	5000	25
Individuals	5000	5000	5000	5001	4291

Results

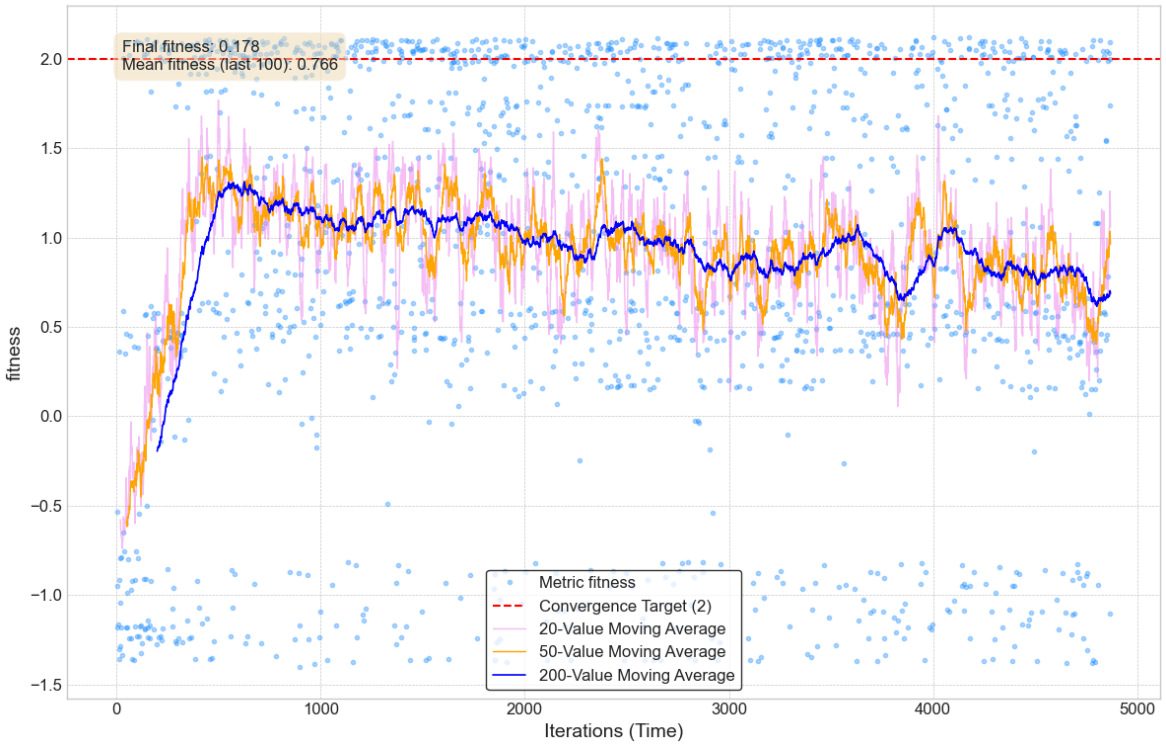
	NSGA-II	NSGA-III	SPEA2	PAES	MOEA/D
Average *fitness*	**1,888**	**1,015**	**0,766**	**0,273**	**1,551**
Winners	**174**	**292**	**220**	**17**	**132**
Optima	**83**	**155**	**125**	**5**	**75**
Success Rate	**48%**	**53%**	**57%**	**29%**	**57%**
File Size (kB)	**267,83**	**258,68**	**276,63**	**417,06**	**251,62**
Compression	**-96,856%**	**-96,964%**	**-96,753%**	**-95,105%**	**-97,047%**
***PEAQ* Score**	**-2,648096**	**-2,651303**	**-2,639016**	**-2,5902**	**-2,7136**
Distortion Index *(IM)*	**-0,767831**	**-0,771335**	**-0,757792**	**-0,704**	**-0,79364**
	Best				***2nd Best***

NSGA-III

As can be seen, it quickly achieved a greater improvement than NSGA-II, but then stagnated in that region, being practically unable to improve during the rest of the cycle.

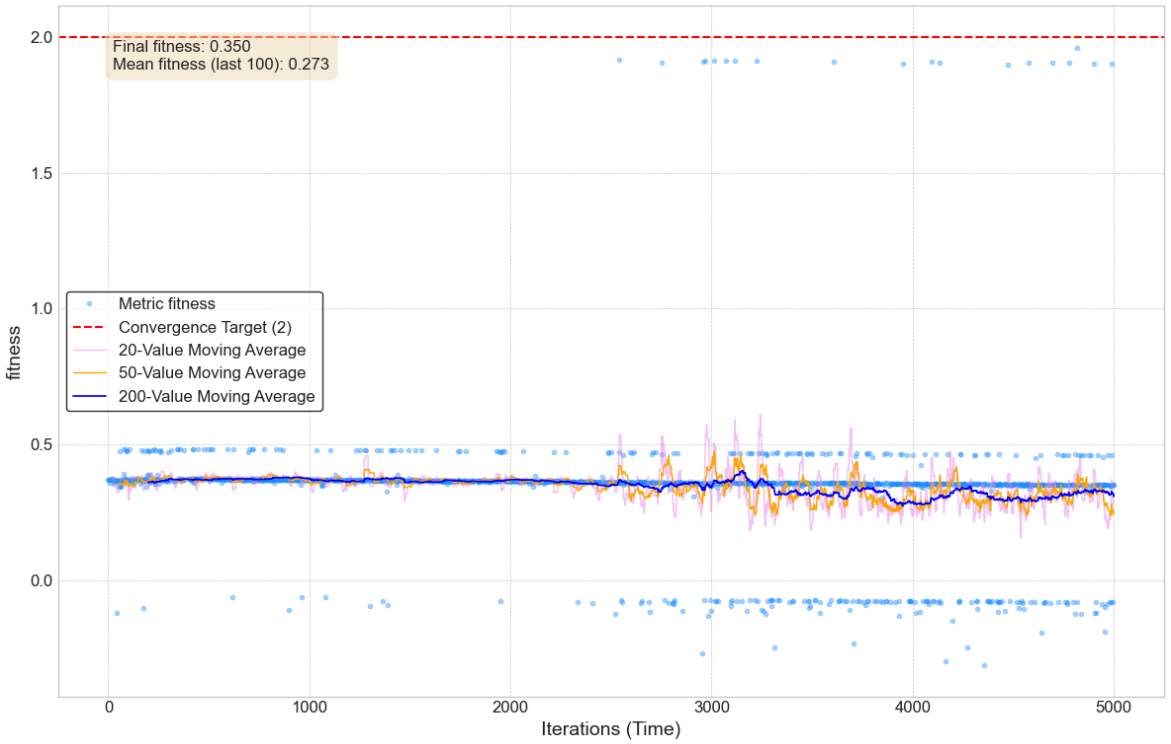
SPEA2

Similarly, SPEA2 achieved a rapid improvement (near individual 500) and then progressively worsened, generating a greater number of individuals in the central region (average *Fitness* of 0.5).

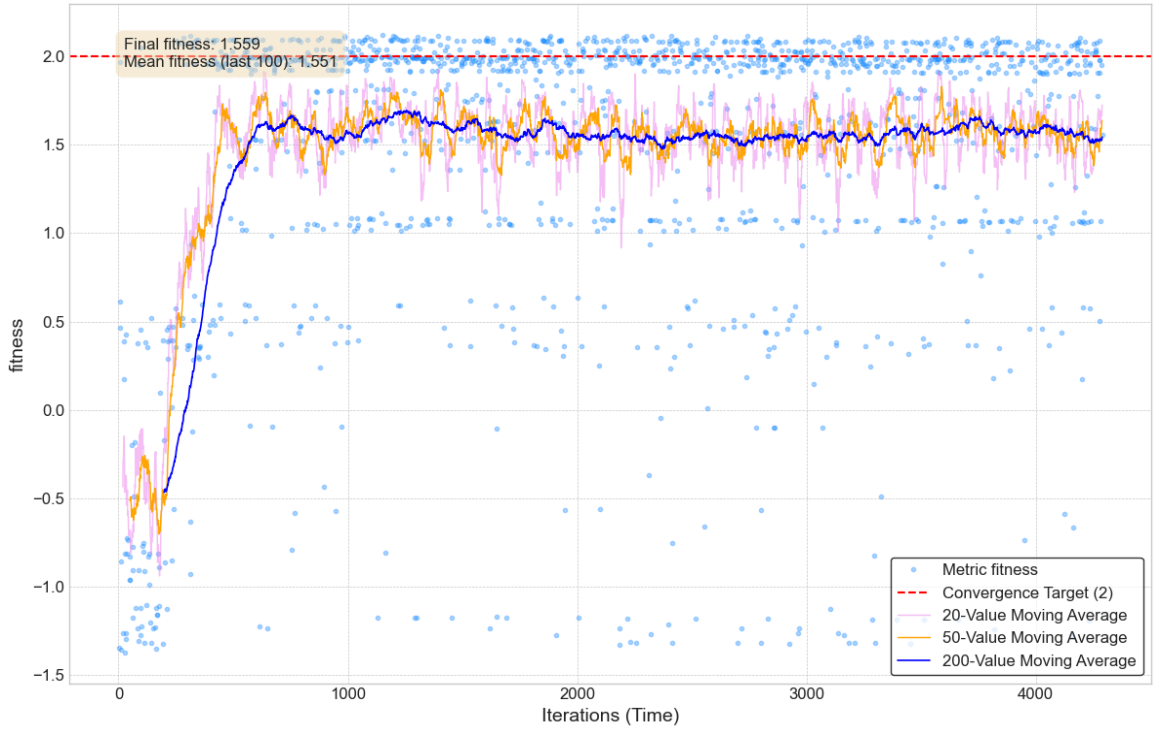


PAES

PAES offered the worst results, as it was seen to be very dependent on the initial conditions, which practically determine the final result, without obtaining significant improvements during the genetic cycle. It was necessary to run it several times to achieve a "convenient" initialization (the one presented here), and yet the individuals were of much worse quality than with the other algorithms.



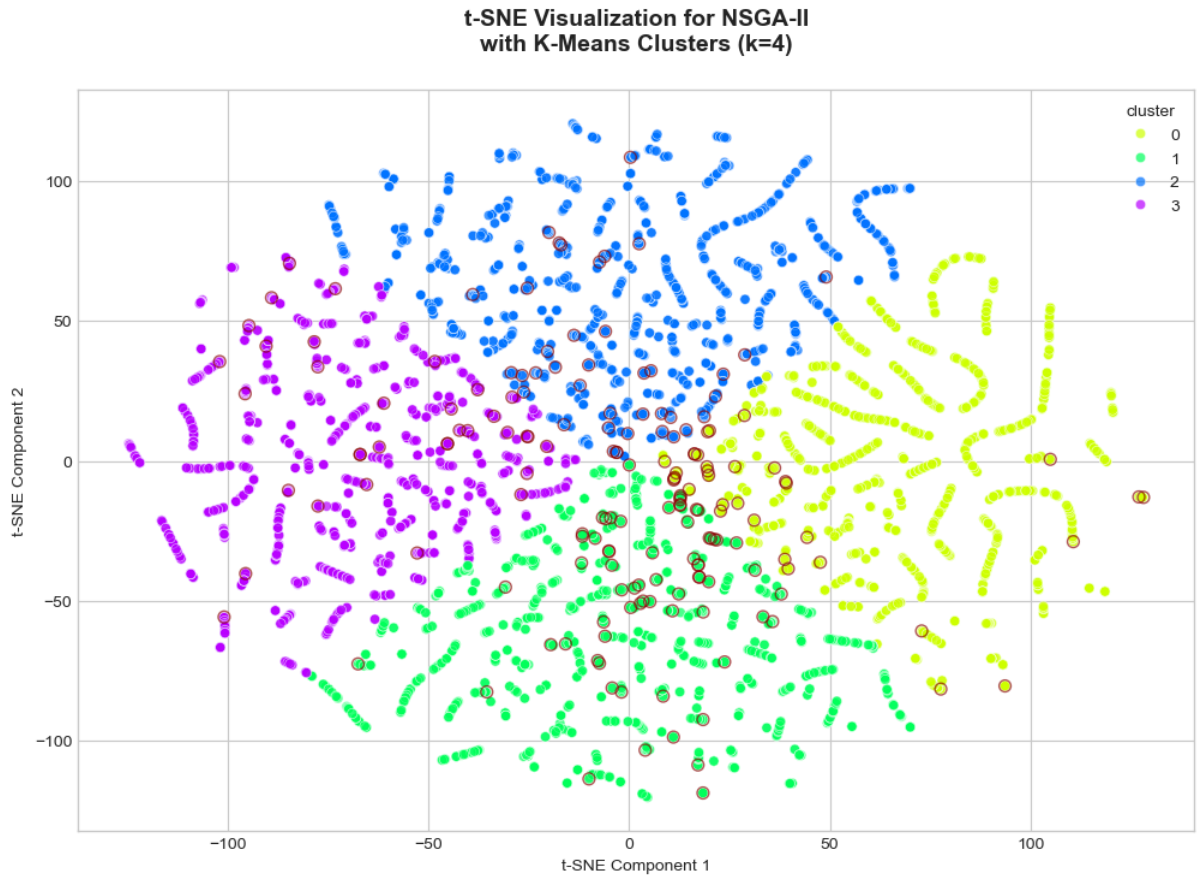
MOEA/D



MOEA/D was much more complex to configure and does not scale very well, as small modifications to the hyperparameters cause an exponential growth in the total number of individuals, which makes it difficult to train in a reasonable time. Despite this, it is observed that it offered better results than the other alternative algorithms, achieving a rapid improvement (near individual 500), and then maintaining average *Fitness values* of 1.5, but generating many optimal individuals (close to the target).

Clustering and Parameter Analysis

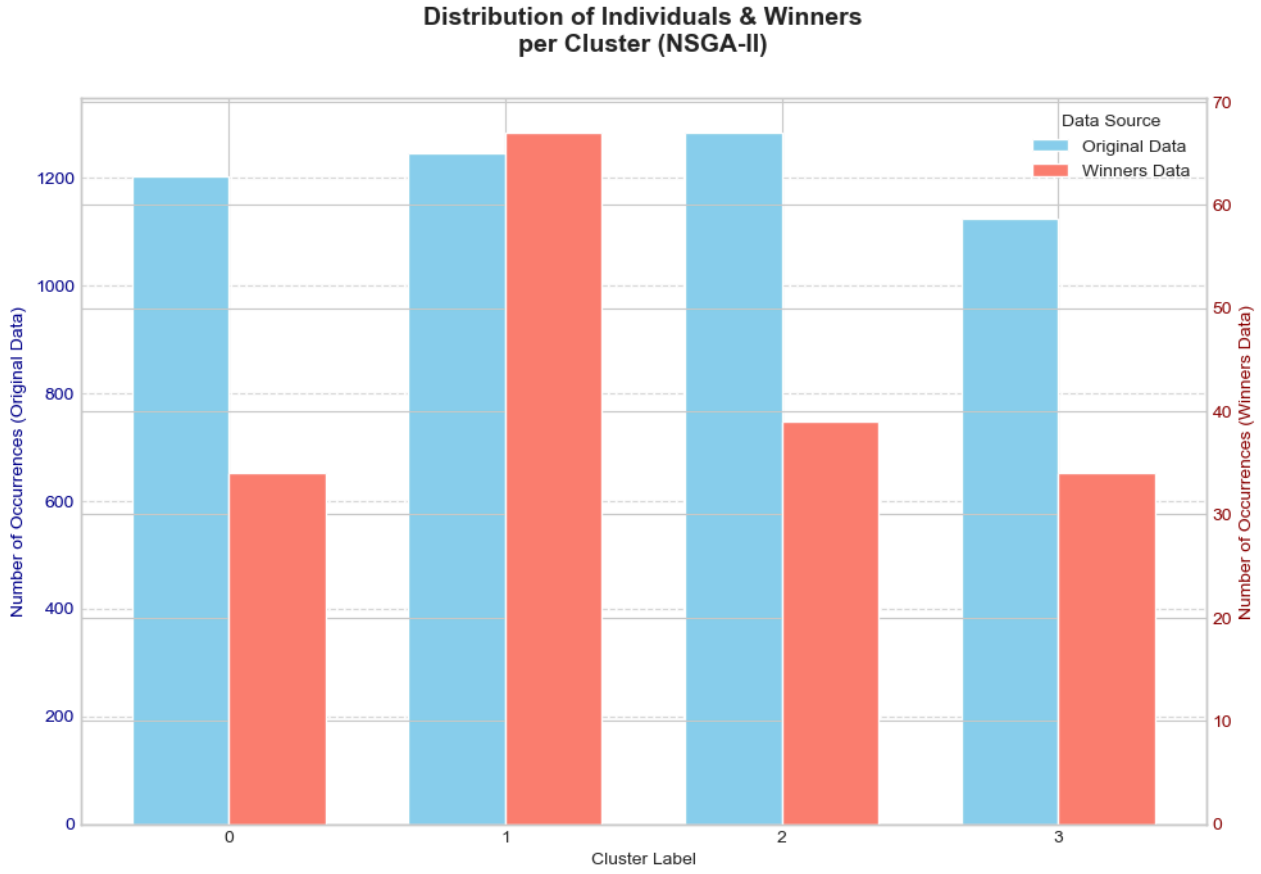
t-SNE was applied, using the *Silhouette* index, to identify clusters within the HOF individuals. In addition, those individuals that meet the previously established condition (*PEAQ* > -2.8) were highlighted with a circle.



In this way, clusters with the highest number of "optimal" individuals are detected, and an analysis of the FFmpeg parameters they share is performed.

Hall of Fame (174 individuals)

- ***Reservoir*:** It is almost always activated (88% of individuals). This suggests that bit control in the reservoir is beneficial for *Fitness* optimization.
- **Sampling frequency *(ar):*** Mostly 44.1 kHz (71%), the standard frequency for CD-quality audio. This demonstrates that the results are viable for industry use.



- **Encoding mode:**
  - **Average Bit Rate (ABR):** was activated for most individuals (72%). This indicates that it is the best encoding mode for the stated objectives.
  - **Variable Bit Rate (VBR):** was used by 15% of the winning individual.
  - **Constant Bit Rate (CBR):** while the remaining 13% used CBR.
- **Bit Rate for ABR/CBR** *(b:a):* Continuous values in the range of 130-170 kbps.
- **VBR audio quality** *(aq):* Highest *fitness* for values 2, 4, and 5.
  - Medium values (4 and 5) show a great balance between quality and compression.
  - Surprisingly, an extremely low value like 2 also achieved a high *fitness* (1.93686). This could indicate that, for certain configurations, a "lower" VBR quality (which could mean a lower average bitrate) can still produce perceptually very good results.
- **Compression level** *(compression_level):*
  - Maximum compression (level 9) is quite frequent and describes 60% of the HOF individuals. This indicates that the algorithm often used this parameter to reduce the size.

- Medium compression (level 4) was used for 21% of the individuals.
- Values between 0 and 3, which indicate minimum compression, were used only 13% of the time.

- **Bit Depth** *(sample_fmt):* 's16p' is the most common format in most individuals (58%). It is widely compatible and has standard quality.

It is also important to highlight the parameters that were not winners:

- ✕ The fact that **no HOF individual uses a 48 kHz** sample rate shows that the resolution increase provided is **not perceptually justified**. This is a common discussion among audiophiles and producers.
- ✕ **Compression level:** Level 5 was never used, and level 6 was used very little.

Most successful cluster (67 individuals)

- **Sample rate** *(ar):* 44.1 kHz for all individuals.
- ***Reservoir:*** Generally activated (85%).
- **Encoding mode:**
  - **Average Bit Rate (ABR):** 80% of the individuals in the cluster used average bit rate encoding,
  - **Variable Bit Rate (VBR):** 16% used variable bit rate,
  - **Constant Bit Rate (CBR):** only 4% used constant bit rate.
- **Bitrate for ABR/CBR (** *(b:a):* Around 148 kbps (80% of individuals).
- **VBR Audio Quality** *(aq): Mostly mid-range values (6 and 5).*
- **Bit Depth** *(sample_fmt):* 's16p', a 16-bit signed integer in planar format (66%).
- **Compression Level** *(compression_level):*
  - 9, high compression (40%).
  - 4, medium compression (30%).

Optimal Individual

FFmpeg Parameters:
{ar': '44100', 'sample_fmt': 's32p', 'compression_level': '4', 'reservoir': '0', 'b:a': '190k'}

- **Sampling Frequency** *(ar):* 44.1 kHz.
- **Bit Depth** *(sample_fmt):* 's32p', 32-bit signed integer, planar format.
- **Encoding Mode:** Constant Bit Rate (CBR).
- **Audio Bitrate** *(b:a):* 190 kbps.
- **Compression Level** *(compression_level):* 4, medium compression.

- **VBR Audio Quality** ***(aq):*** Disabled.
- ***Reservoir:*** Disabled.

With these parameters, a ***Fitness value*** of **2.127855** was achieved during the evolutionary cycle, composed of a ***PEAQ*** of **-2.613**, an ***IM*** of **-0.728**, and an **effective compression** of **-96.859%.**

Finally, a complete music fragment was processed with the same parameters to verify if they could generalize to the full track. Starting from a **22.3 MB WAV file**, it was compressed to a **698 kB MP3 format**, a **-96.867% reduction**, with ***PEAQ*** metrics at **-2.593** and ***IM*** at **-0.707.** It was verified that similar values to the optimal individual were obtained. Processing 4 minutes of music took only 5.24 seconds.

For informational purposes, when comparing both files auditorily — with professional headphones — no significant quality difference or distortion is perceptible.

Conclusions

1) CD audio quality (44.1 kHz) and the standard format (s16p) are optimal.
2) High bitrate values (between 130 and 170 kbps) tend to achieve high *Fitness* scores.
3) ABR (Average Bit Rate) is the dominant encoding mode (72% in the Hall of Fame, 80% in the most successful cluster), suggesting it allows for the best balance between file size and perceived quality for the stated objectives.
4) However, excellent results can also be obtained with VBR adjusted to medium quality, or even with CBR, by setting an intermediate compression level.
5) If the goal is to maximize file size reduction, aggressive compression (level 9) is preferred. Conversely, if the goal is to preserve quality, a medium value (level 4) is preferable.
6) The activation of the bit "reservoir" for bit control was a key feature for efficiency (88% in the Hall of Fame, 85% in the most successful cluster), as it allows for bit savings in simpler audio passages.

Paradoxically, the individual selected as “optimal” does not use any of the most common parameters found in the HOF. This suggests that, while ABR is generally preferred, a CBR with a relatively high bitrate and a higher-depth sample format can achieve superior perceptual quality along with very high compression.